

EXPERIMENT - 2

Stages of C Compilation and Execution

I. Objective

To demonstrate the stages of C program compilation using the `-save-temps` flag and to write, compile, and execute a C program that calculates the area of a circle.

2. Theory: Stages of C Compilation

The process of converting a C source code file into an executable binary involves four primary stages:

2.1 Preprocessing

The preprocessor removes comments, expands macros (like `#define`), and includes the contents of header files (like `stdio.h`). It generates an intermediate `.i` file.

2.2 Compilation

The compiler translates the preprocessed C code into assembly language instructions specific to the target architecture. It generates a `.s` file.

2.3 Assembly

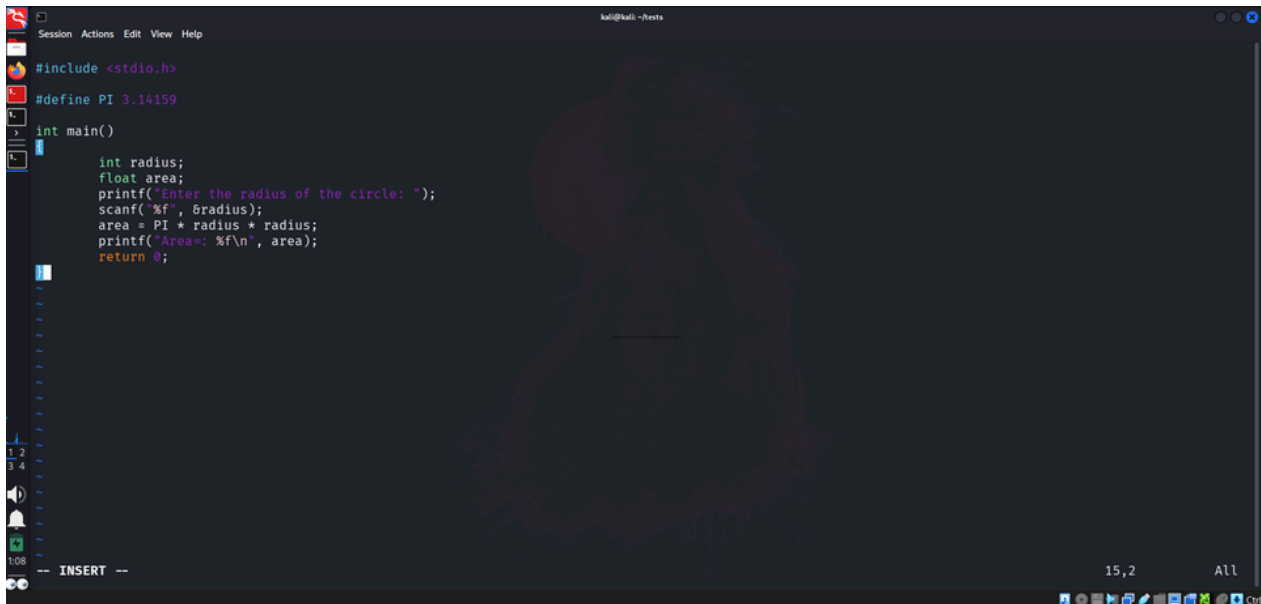
The assembler converts the assembly language code into machine-level object code (binary). It generates a `.o` file.

2.4 Linking

The linker combines the generated object code with standard library files to resolve function calls (e.g., `printf`, `scanf`) and creates the final executable file

3. The C Program: Area of a Circle

The following C program calculates the area of a circle using the standard formula $\text{Area} = \pi r^2$

A screenshot of a text editor window titled 'kali@kali:~/tests'. The editor contains a C program to calculate the area of a circle. The code includes a header file, defines a constant for PI, and has a main function that prompts the user for a radius, calculates the area, and prints it. The code is as follows:

```
#include <stdio.h>

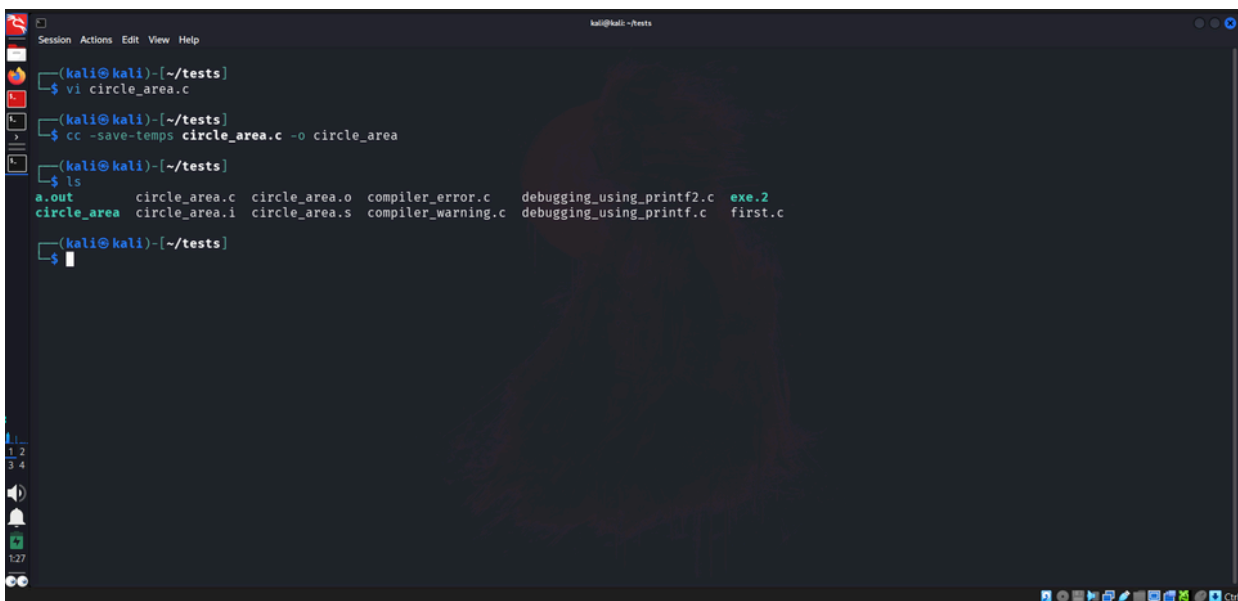
#define PI 3.14159

int main()
{
    int radius;
    float area;
    printf("Enter the radius of the circle: ");
    scanf("%f", &radius);
    area = PI * radius * radius;
    printf("Area=: %f\n", area);
    return 0;
}
```

4. Demonstrating Compilation Stages

To observe all the intermediate files generated during the compilation process, we use the `-save-temps` flag with `cc` compiler.

4.1 Compiling with `-save-temps`

A screenshot of a terminal window titled 'kali@kali:~/tests'. It shows the steps to compile a C program and list the generated files. The commands and their outputs are:

```
(kali@kali)-[~/tests]
└─$ vi circle_area.c
(kali@kali)-[~/tests]
└─$ cc -save-temps circle_area.c -o circle_area
(kali@kali)-[~/tests]
└─$ ls
a.out      circle_area.c  circle_area.o  compiler_error.c  debugging_using_printf2.c  exe.2
circle_area  circle_area.i  circle_area.s  compiler_warning.c  debugging_using_printf.c  first.c
```

4.2 Analysis of Intermediate Files

- **circle_area.i (Preprocessed Code):** Opening this file reveals that the code has expanded significantly due to the inclusion of `stdio.h`. Furthermore, the comment has been removed, and the `PI` macro inside the calculation has been explicitly replaced with `3.14159`.
- **circle_area.s (Assembly Code):** This file contains the assembly-level instructions generated by the compiler. Below is a captured view of the assembly file content:

```

Session Actions Edit View Help
.file "circle_area.c"
.text
.section .rodata
.align 8

.LC0:
.string "Enter the radius of the circle: "
.LC1:
.string "%f"
.LC3:
.string "Area=: %f\n"
.text
.globl main
.type main, @function

main:
.LFB0:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
subq $16, %rsp
leaq .LC0(%rip), %rax
movq %rax, %rdi
movl $0, %eax
call printf@PLT
leaq -8(%rbp), %rax
leaq .LC1(%rip), %rdx
movq %rax, %rsi
movq %rdx, %rdi
movl $0, %eax
call __isoc23_scanf@PLT
movl -8(%rbp), %eax
"circle_area.s" 65L, 11608

```

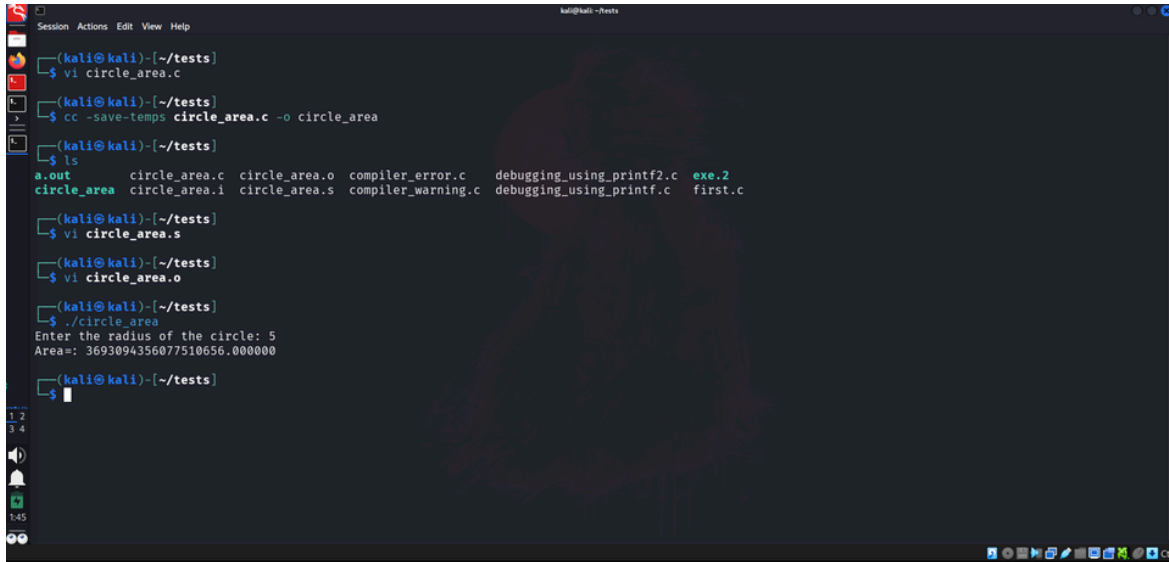
- **circle_area.o (Object Code / Machine Code):** This contains raw binary machine code. Attempting to view it with a standard text editor yields unreadable characters. To inspect it, a hexadecimal dump utility like `hexdump` is used. Below is a captured hex dump of the machine code:

[illegible]

- circle_area (Executable): The final linked binary file ready for execution by the operating system.

5. Program Execution

Executing the final compiled binary to calculate the area.



```
(kali@kali)~/tests
$ vi circle_area.c

(kali@kali)~/tests
$ cc -save-temps circle_area.c -o circle_area

(kali@kali)~/tests
$ ls
a.out      circle_area.c  circle_area.o  compiler_error.c  debugging_using_printf2.c  exe.2
circle_area  circle_area.i  circle_area.s  compiler_warning.c  debugging_using_printf.c  first.c

(kali@kali)~/tests
$ vi circle_area.s

(kali@kali)~/tests
$ vi circle_area.o

(kali@kali)~/tests
$ ./circle_area
Enter the radius of the circle: 5
Area=: 3693094356077510656.000000

(kali@kali)~/tests
$
```

6. Conclusion:

Conclusion: The stages of compilation were successfully analyzed using intermediate files, and the logic to calculate the area of a circle was executed properly.